# Conception de l'API REST

> **Important**
>
> Vous ne serez pas autorisés à utiliser Postman pendant l'examen car l'outil intègre un moteur IA. Nous vous recommandons dès lors de vous familiariser avec l'outil REST Client (fichiers `.http`) pour IntelliJ pour tester vos *endpoints*.

## Structuration des fichiers backend

Voici une proposition de structuration des fichiers de votre backend :

```
05.<yy>_ep_<bbbb>.sql   -- Fonctions correspondant aux endpoints de l'API
regroupées par fonctionnalité
                        -- où <bbbb> est un nom court décrivant la
fonctionnalité (ex: reservations, users, ...)
                        -- et <yy> est un numéro de séquence pour la
fonctionnalité
```

## Généralités

- **Méthodes HTTP** : Tous les endpoints RPC acceptent GET et POST
  - **Recommandation** :
    - Utilisez **GET** pour les opérations de lecture simples (peu de paramètres, valeurs courtes)
    - Utilisez **POST** pour les opérations de lecture complexes (nombreux paramètres, strings longues) et toutes les modifications
    - PostgREST bloque automatiquement les modifications de données via GET
  - **Paramètres** :
    - Avec GET : passés en query string ( `?param1=value1&param2=value2` )
    - Avec POST : passés dans le body JSON
- Les paramètres optionnels peuvent être omis ou envoyés comme `null`
- Les dates/heures sont au format ISO 8601 (ex: `"2024-12-20T19:00:00"` )
- Les heures sont au format `HH:MM:SS` (ex: `"19:00:00"` )

- Les dates sont au format `YYYY-MM-DD` (ex: `"2024-12-20"`)
- Les tableaux sont représentés en JSON comme des listes : `[1, 2, 3]`
- Les erreurs sont retournées avec un message descriptif dans le corps de la réponse

## Nommage des endpoints

- **Lecture**
  - Un élément : `get_<ressource>` au singulier (`get_restaurant`, `get_reservation`, `get_user`).
  - Liste : `get_<ressources>` au pluriel (`get_restaurants`, `get_reservations`, `get_services`, `get_tables`).
- **Écriture**
  - Création + mise à jour : un seul endpoint `save_<ressource>` avec un paramètre d'ID optionnel (`reservation_id`, `service_id`, `table_id`). S'il est présent → mise à jour, sinon → création.
  - Suppression : `delete_<ressource>` (`delete_service`).
- **Changements d'état**
  - Un endpoint par transition : `confirm_reservation`, `cancel_reservation`, `complete_reservation` (verbe + entité), plutôt qu'un seul "update status".
- **Vérifications / règles métier**
  - Disponibilité : `check_<quoi>_available` (`check_email_available`, `check_full_name_available`).
  - Droits / possibilité d'action : `can_<action>` (`can_create_reservation`, `can_create_service`, `can_delete_services`).
  - Impact (pour vérifier en amont l'impact qu'aurait une action) : `check_<quoi>` (`check_service_reservations`).

## Paramètres

- **Identifiants** : nommer en `_id` cohérent avec l'entité (`restaurant_id`, `reservation_id`, `user_id`, `service_id`, `table_id`).
- **Create vs update** : pour un `save_*`, l'ID de l'entité est **optionnel** ; s'il est absent → création, sinon → mise à jour (et les champs requis peuvent dépendre du mode, ex. `restaurant_id` requis en création).
- **Exclusion** : paramètres dédiés pour exclure une entité du calcul (`exclude_reservation_id`, `exclude_service_id`) ou de la vérification (`user_id` dans

`check_email_available` pour s'exclure soi-même).

- **Listes** : filtres explicites ( `search_filter` , `limit_count` pour la pagination "limit + 1").

## Forme des réponses

- **Types métier réutilisables** : réponses typées via des "types composites" (voir slides Postgrest).
- **Listes** : tableaux de ces types.
- **Vérifications booléennes** : retourner un booléen ( `true` / `false` ) pour `check_*_available` et `can_*` .
- **Pas de corps** : code **204** quand il n'y a rien à retourner (ex. `signup` ).
- **Après mutation** : renvoyer l'entité mise à jour (même type que le détail) pour éviter un second appel (ex. `save_reservation` , `cancel_reservation` , `confirm_reservation` → `reservation_info` ).
- Limiter le nombre d'appels API en imbriquant les types composites et les listes pour créer des structures de données complexes. Voir slides Postgrest et exemple `get_restaurant` plus loin.

## Contrôle d'accès

**3 niveaux de vérification** :

- Vérification de l'authentification (public ou authentifié)
- Vérification du rôle (client, manager)
- Vérification du lien avec la ressource (exemples : client = sa réservation, manager = son restaurant)

## Exemple de endpoint

Obtenir les détails d'un restaurant : `/rpc/get_restaurant`

**Paramètre :**

- `restaurant_id` (integer, requis) : ID du restaurant

**Exemple de réponse :**

La réponse est un type composite `restaurant_details` qui contient :

| restaurant_details |
| --- |

```
{
  "id": 1,
  "name": "Le Bon Restaurant",
  "address": "Rue de la Paix 1",
  "city": "Bruxelles",
  "phone": "+32123456789",
  "description": "Un excellent restaurant",
  "rating": 4.5,
  "price_range": 2,
  "slot_duration": 30,
  "tables": [table_info, ...],
  "services": [service_info, ...]
}
```

`tables` et `services` sont des tableaux de types composites `table_info` et `service_info` respectivement.

**table_info**

```
{
  "id": 1,
  "restaurant": 1,
  "table_number": 1,
  "capacity": 4
}
```

**service_info**

```
{
  "id": 1,
  "restaurant": 1,
  "day_of_week": 1,
  "start_time": "12:00:00",
  "end_time": "14:00:00"
}
```

**Contrôle d'accès :** être authentifié en tant que client ou manager du restaurant.